

IA 26

Guide, Module 26

L'IA pour les codeurs

Créer des applications et du code avec l'IA le Vibe Coding

Décrire ce que l'on veut, et le voir prendre forme la dernière barrière qui tombe.

Matthieu Jardin, Marketing AI

Édition 2026

Vous pouvez obtenir une copie du livre en format .pdf en [cliquant-ici](#)

Sommaire

Préface : Pourquoi ce guide ?

Introduction : Le défi de l'idée qu'on ne peut pas réaliser soi-même

Partie 1 : Contexte et enjeux : de la barrière du code à la création par l'intention

Partie 2 : Les concepts fondamentaux

Partie 3 : La méthode pas à pas : créer son premier projet avec l'IA

Partie 4 : Cas pratiques

Partie 5 : Outils, templates et checklists

Les 7 pièges à éviter

Synthèse et plan d'action en 7 jours

Pour aller plus loin

À propos de la FORMATION IA 26

Préface : Pourquoi ce guide ?

Voici le dernier guide de la série. Pendant vingt-cinq modules, vous avez appris à utiliser l'IA, à la connecter, à l'automatiser, à l'intégrer à vos outils. Ce module aborde la transformation la plus spectaculaire de toutes : l'IA ne se contente plus d'utiliser le code elle l'écrit. Et elle l'écrit à partir de ce que vous lui décrivez, en langage ordinaire. C'est ce qu'on appelle désormais le « vibe coding » : créer un programme en exprimant son intention, plutôt qu'en maîtrisant une syntaxe.

Ce guide est le vingt-sixième et dernier d'une série de 26 guides qui accompagnent le programme FORMATION IA 26. Il refferme la formation, et il le fait sur une note forte : la barrière la plus ancienne de l'informatique « il faut savoir coder pour créer » est en train de s'abaisser. Le vibe coding ne fait pas de tout le monde un développeur. Mais il met la création d'outils, de scripts, de petites applications à portée de quiconque sait décrire clairement ce qu'il veut et vérifier ce qu'il obtient.

Vous y trouverez : ce qu'est le vibe coding et ce qu'il déplace ; l'IA comme binôme de programmation ; les assistants de code et les plateformes de création ; le dialogue itératif qui fait avancer un projet ; pourquoi le code produit reste du code, à comprendre et à tester ; et la vigilance essentielle un programme qui « marche » n'est pas un programme qu'on maîtrise. L'objectif est clair : que vous refermiez ce guide, et cette formation, capable de transformer une idée en un premier outil concret et de le faire avec lucidité.

« Hier, créer un logiciel demandait de connaître un langage. Aujourd'hui, cela demande surtout de savoir décrire son intention et de vérifier le résultat. » Principe directeur du Module 26

Introduction : Le défi de l'idée qu'on ne peut pas réaliser soi-même

Presque tout le monde a, un jour, eu une idée d'outil. Un petit programme qui automatiserait une tâche, une application qui rendrait service, un script qui ferait gagner du temps. Et presque tout le monde s'est heurté au même mur : « je ne sais pas coder ». Restaient alors deux options, toutes deux coûteuses : apprendre à programmer des mois, des années ou payer quelqu'un pour le faire. Entre l'idée et sa réalisation, il y avait un fossé. Ce module traite de la façon dont l'IA comble ce fossé.

Le frein, ici, est une croyance profondément installée : « la création de logiciels n'est pas pour moi ». Le vibe coding ne supprime pas tout effort il déplace l'effort. Il ne demande plus de maîtriser une syntaxe ; il demande de savoir formuler une intention claire, dialoguer avec l'IA, et vérifier ce qu'elle produit. Trois questions structurantes guident l'intégralité des pages qui suivent :

1. Qu'est-ce que le vibe coding, et qu'est-ce qu'il change concrètement ?
2. Quels outils permettent de créer du code avec l'IA, et comment s'en servir ?
3. Comment créer quelque chose qui fonctionne vraiment et que l'on maîtrise ?

Ces trois questions correspondent à trois corpus : la nature et la portée du vibe coding, les outils et la méthode pour créer avec l'IA, et l'usage lucide et responsable de ce qu'on produit. Chaque corpus est traité dans la suite du guide avec ses concepts-clés, ses applications concrètes et ses pièges typiques.

Partie 1 : Contexte et enjeux : de la barrière du code à la création par l'intention

1.1 La limite de la barrière du code

Depuis l'origine de l'informatique, créer un logiciel supposait de parler la langue de la machine un langage de programmation, avec sa syntaxe rigoureuse, ses règles, ses années d'apprentissage. Cette exigence a longtemps tracé une frontière nette : d'un côté ceux qui pouvaient créer, de l'autre ceux qui ne pouvaient qu'utiliser. La grande majorité des professionnels se trouvait du second côté, avec des idées mais sans les moyens de les réaliser eux-mêmes.

Le vibe coding s'attaque à cette frontière. Son principe : l'IA maîtrise déjà les langages de programmation ; il suffit donc de lui décrire, en langage ordinaire, ce que l'on veut, et elle produit le code correspondant. La compétence rare n'est plus la syntaxe c'est la clarté de l'intention et la capacité à vérifier. La barrière du code ne disparaît pas tout à fait, mais elle s'abaisse considérablement. La création par l'intention remplace la création par la syntaxe.

1.2 Les douleurs typiques de qui veut créer sans savoir coder

Avant de progresser, il faut nommer ce qui freine. Voici les difficultés les plus fréquentes face à la création d'outils numériques :

- **L'idée qui reste lettre morte.** Avoir un projet d'outil utile, et le laisser de côté faute de savoir le réaliser.
- **La dépendance totale.** Devoir solliciter et payer quelqu'un pour la moindre création ou modification.
- **Le coût de l'apprentissage.** Croire que la seule voie est d'apprendre un langage de programmation, et y renoncer.
- **La méconnaissance des outils.** Ignorer que des assistants et des plateformes permettent désormais de créer en décrivant.
- **Le vibe coding sans recul.** À l'inverse, générer du code sans le comprendre ni le tester, et le croire fiable parce qu'il « semble marcher ».
- **La confusion sur ce qui est à portée.** Ne pas distinguer ce qu'on peut raisonnablement créer soi-même de ce qui relève encore d'un professionnel.

« *Le vibe coding ne vous dispense pas de réfléchir. Il vous dispense d'apprendre une syntaxe pas de comprendre ce que vous construisez.* » Principe directeur du Module 26

1.3 Le pari de ce guide

Le pari de ce guide, et plus largement du programme FORMATION IA 26, est que tout professionnel peut désormais créer ses premiers outils numériques à condition de garder la tête froide. Le vibe coding n'est ni magique ni sans risque : c'est une compétence, avec sa méthode et ses pièges. Ce guide fournit cette méthode : choisir une idée à sa portée, décrire clairement, itérer, tester réellement, et comprendre ce qu'on a produit. Il referme la formation en transformant tout ce qui précède en capacité de création. Les parties suivantes le détaillent.

À RETENIR

Le vibe coding déplace la compétence rare. Ce n'est plus la syntaxe d'un langage l'IA la maîtrise mais la clarté de l'intention et la rigueur de la vérification. C'est aussi le module qui referme la formation : après avoir appris à utiliser, connecter et automatiser l'IA, vous apprenez à créer avec elle. La dernière barrière s'abaisse à condition de ne jamais confondre « cela semble marcher » et « je maîtrise ce que j'ai construit ».

Partie 2 : Les concepts fondamentaux

Cette partie présente les dix notions qui permettent de comprendre et d'aborder le vibe coding. Aucune ne demande de savoir programmer au préalable il s'agit de comprendre une pratique, pas d'écrire du code à la main. Une fois ces notions assimilées, vous saurez créer avec l'IA, et le faire avec lucidité.

2.1 Le vibe coding : qu'est-ce que c'est

Le vibe coding désigne une façon de créer du code en décrivant à l'IA, en langage ordinaire, ce que l'on veut obtenir plutôt qu'en écrivant soi-même chaque ligne. Le terme s'est popularisé début 2025, à l'initiative du chercheur Andrej Karpathy, pour nommer cette pratique : on se concentre sur l'intention, sur la « vibe » du résultat voulu, et l'on laisse l'IA produire la syntaxe. L'humain dirige et décide ; l'IA écrit. C'est un changement de posture autant que d'outil : on passe d'auteur du code à concepteur et superviseur du code.

2.2 De la syntaxe à l'intention : ce que le vibe coding déplace

Le cœur du vibe coding tient en un déplacement. Programmer « classiquement », c'est traduire une intention en une syntaxe précise et l'essentiel de la difficulté est dans cette traduction. Le vibe coding confie la traduction à l'IA. Ce qui reste à l'humain, c'est l'amont formuler une intention claire, complète, sans ambiguïté et l'aval vérifier que ce qui sort correspond à ce qui était voulu. La compétence ne disparaît pas : elle se déplace vers les deux bouts de la chaîne, là où le jugement humain est irremplaçable.

2.3 L'IA comme binôme de programmation

La meilleure image pour comprendre le vibe coding est celle du binôme. L'IA n'est pas un distributeur de code : c'est un partenaire avec qui l'on dialogue. On lui explique l'objectif, elle propose ; on regarde, on réagit, on précise ; elle ajuste. Comme avec un binôme humain, la qualité du résultat dépend de la qualité de l'échange : des consignes floues donnent un travail flou. Et comme avec un binôme, c'est vous qui restez responsable du résultat le binôme aide, il ne décharge pas de la responsabilité.

2.4 Les assistants de code intégrés

Une première famille d'outils : les assistants de code intégrés à l'environnement de travail des développeurs. Ils complètent, suggèrent, génèrent du code au fil de l'écriture, et permettent de dialoguer en langage naturel pour faire évoluer un projet. GitHub Copilot, Cursor, Claude Code en sont des exemples représentatifs. Ces outils s'adressent surtout à qui code déjà, ou souhaite apprendre : ils accélèrent et assistent, mais supposent un environnement de développement. Ils sont la porte d'entrée naturelle pour le codeur qui veut gagner en vitesse.

2.5 Les plateformes de création par l'IA

Une seconde famille : les plateformes qui génèrent une application complète à partir d'une description. On y décrit ce que l'on veut, et la plateforme produit un résultat fonctionnel souvent une application web que l'on fait ensuite évoluer par le dialogue. Ces plateformes visent justement le non-développeur : elles masquent l'environnement technique et permettent de créer un prototype, un outil simple, une première version d'une idée, sans rien installer. C'est la porte d'entrée naturelle pour qui n'a jamais codé et veut concrétiser une idée.

2.6 Le dialogue itératif : décrire, voir, corriger

Le vibe coding n'est presque jamais affaire d'une seule demande. C'est un cycle : on décrit, l'IA produit, on regarde le résultat, on identifie ce qui ne va pas ou ce qui manque, on le redit et le cycle reprend. Chaque tour rapproche du but. C'est exactement la logique itérative déjà rencontrée au fil de la formation, appliquée à la création de code. Accepter ce caractère itératif est essentiel : un projet de vibe coding ne se juge pas sur le premier jet, mais sur ce vers quoi le dialogue le fait converger.

2.7 Le code reste du code : lisibilité, test, maintenance

Un point que l'enthousiasme fait souvent oublier : ce que l'IA produit est du code véritable, avec toutes les exigences du code. Il doit fonctionner, mais aussi être testé sur de vrais cas, rester compréhensible, et pouvoir être maintenu et corrigé dans le temps. Un code qui « marche » au premier essai peut comporter des défauts invisibles erreurs sur des cas particuliers, failles, choix qui rendront les évolutions difficiles. Le vibe coding ne supprime aucune de ces réalités : il change qui écrit le code, pas la nature du code.

2.8 Vibe coding et MCP : l'IA codeur connectée

Le Module 26 prolonge directement le Module 25. Une IA qui code est d'autant plus utile qu'elle est connectée à l'environnement réel vos fichiers, vos projets, vos outils. C'est précisément ce que permet le MCP : donner à l'IA codeur un accès standardisé au contexte sur lequel elle travaille. Vibe coding et MCP se rejoignent ainsi naturellement : l'un fait écrire le code par l'IA, l'autre branche cette IA sur le réel. C'est l'aboutissement du dernier bloc de la formation l'IA qui crée, et qui crée en prise avec votre environnement.

2.9 Pour qui : le non-codeur qui crée, le codeur qui accélère

Le vibe coding sert deux publics, différemment. Pour le non-codeur, c'est une ouverture : la possibilité de concrétiser des idées d'outils simples, de prototyper, de ne plus dépendre entièrement d'un tiers pour la moindre création. Pour le codeur, c'est une accélération : déléguer le répétitif, explorer plus vite, se concentrer sur la conception et les choix difficiles. Dans les deux cas, le vibe coding ne remplace pas le jugement il amplifie ce que chacun peut faire, à partir de son niveau.

2.10 Vigilance : le code qui « marche » n'est pas le code qu'on maîtrise

C'est le point le plus important du module, et un bon point final pour toute la formation. Le vibe coding rend la création si fluide qu'il fait courir un risque précis : confondre « cela fonctionne » avec « je maîtrise ce que j'ai construit ». Or un programme peut sembler marcher tout en cachant des erreurs, des failles de sécurité, des fragilités. Quatre réflexes en découlent. D'abord, rester dans son périmètre : créer des outils simples et à faible enjeu tant qu'on débute, et confier à de vrais professionnels ce qui touche à des données sensibles ou à des systèmes critiques. Ensuite, toujours tester réellement, sur des cas variés, y compris particuliers. Puis, chercher à comprendre ce qu'on a produit, au moins dans ses grandes lignes refuser la boîte noire. Enfin, ne jamais mettre en usage réel ce qu'on n'a pas vérifié. Le vibe coding donne un pouvoir de création ; il demande, en retour, une honnêteté sur ce que l'on maîtrise vraiment.

Partie 3 : La méthode pas à pas : créer son premier projet avec l'IA

Cette partie traduit les concepts en séquence opérationnelle. Suivez-la dans l'ordre : chaque étape construit la suivante. L'ensemble se met en place en une semaine, à raison d'une heure par jour l'objectif étant de créer un premier outil simple et de le maîtriser, non de bâtir un logiciel ambitieux.

Étape 1 : Choisir une idée simple et utile

Commencez petit. Repérez une idée d'outil simple, à faible enjeu, qui vous rendrait service un script qui automatise une tâche, une petite application qui résout un besoin précis. Évitez, pour un premier projet, tout ce qui touche à des données sensibles ou à des systèmes importants. L'objectif n'est pas l'ambition, c'est l'apprentissage de la méthode sur un terrain sûr.

Étape 2 : Choisir l'outil adapté

Selon votre profil et votre idée, choisissez la famille d'outils. Vous n'avez jamais codé et voulez concrétiser une idée sans rien installer : une plateforme de création par l'IA. Vous codez déjà un peu, ou voulez apprendre en faisant : un assistant de code intégré. Choisir l'outil en fonction de son point de départ évite de se décourager devant une marche trop haute.

Étape 3 : Décrire clairement ce que vous voulez

Tout le vibe coding repose sur cette étape. Avant de demander, mettez votre intention au clair : que doit faire l'outil, exactement ? Pour qui ? Dans quels cas ? Décrivez-le à l'IA de façon précise et complète comme vous l'expliqueriez à un binôme qui ne connaît rien de votre contexte. Une intention floue produit un résultat flou ; une intention claire est déjà la moitié du travail.

Étape 4 : Construire par itérations décrire, voir, corriger

Ne visez pas le résultat parfait du premier coup. Lancez une première version, regardez ce que l'IA a produit, identifiez ce qui ne va pas ou ce qui manque, et redites-le. Avancez tour par tour : chaque itération précise et améliore. C'est le rythme naturel du vibe coding un dialogue qui converge, pas une commande unique.

Étape 5 : Tester réellement sur de vrais cas

Ne vous fiez jamais à l'impression que « cela marche ». Éprouvez votre outil sur des cas réels et variés, y compris des cas particuliers et des situations limites. C'est le test, et lui seul, qui révèle les défauts qu'un coup d'œil rapide ne voit pas. Tant que le test n'est pas concluant, le projet n'est pas terminé il est en cours.

Étape 6 : Comprendre ce que vous avez produit

Avant d'utiliser votre outil, cherchez à comprendre ce qu'il fait au moins dans ses grandes lignes. Demandez à l'IA de vous l'expliquer simplement : quelles sont les parties, à quoi servent-elles, où sont les points sensibles. Refuser la boîte noire n'est pas une option de confort : c'est ce qui vous permettra de le corriger, de le faire évoluer, et de savoir ce que vous pouvez en attendre.

Étape 7 : Étendre du script à l'outil, et vers la maîtrise

Une fois un premier projet réussi et compris, étendez par paliers. Reprenez-le pour l'améliorer, lancez-en un deuxième un peu plus ambitieux, reliez votre IA codeur à votre environnement via le MCP du Module 25. Et, si le goût y est, profitez-en pour apprendre réellement quelques bases de programmation le vibe coding est une excellente porte d'entrée. Mais chaque palier garde la même règle : tester, comprendre, rester dans son périmètre de maîtrise.

MISE EN PRATIQUE Le premier outil, testé et compris

Choisissez une idée d'outil simple, utile et à faible enjeu. Étape 1 : choisissez la famille d'outils adaptée à votre point de départ. Étape 2 : décrivez votre intention de façon claire et complète. Étape 3 : construisez par itérations décrire, voir, corriger. Étape 4 : testez sur de vrais cas, y compris particuliers. Étape 5 : faites-vous expliquer ce que vous avez produit, et vérifiez que vous le comprenez. Mesurez : votre première idée d'outil est-elle devenue quelque chose de concret, que vous maîtrisez ?

Partie 4 : Cas pratiques

Trois illustrations concrètes pour ancrer la méthode dans des situations réelles. Chaque cas part d'une situation initiale, pose un diagnostic, puis détaille l'application.

Cas 1 : L'indépendant qui crée son premier petit outil

Situation : Karim, indépendant, refait chaque semaine la même manipulation fastidieuse de fichiers. Il a souvent pensé qu'un petit script réglerait le problème mais il ne sait pas coder, et n'a jamais franchi le pas.

Diagnostic : Karim a une idée simple, précise et à faible enjeu. C'est le projet type d'une première expérience de vibe coding le terrain idéal pour apprendre la méthode.

Application : Karim choisit un outil adapté à son point de départ il n'a jamais codé. Il décrit précisément ce que le script doit faire : quels fichiers, quelle manipulation, quel résultat attendu. Il procède par itérations : première version, essai, correction. Il teste sur ses fichiers réels, y compris des cas un peu particuliers. Puis il se fait expliquer ce que fait le script, dans les grandes lignes. Karim a désormais un outil qui lui fait gagner du temps chaque semaine et il l'a compris assez pour le corriger le jour où il faudra.

Cas 2 : L'équipe qui prototype une idée avant de la confier à des développeurs

Situation : Une équipe a une idée d'application. Elle hésite à engager un projet de développement coûteux sans savoir si l'idée tient vraiment la route une fois entre les mains des utilisateurs.

Diagnostic : L'enjeu de l'équipe n'est pas de livrer un produit fini, mais de tester une idée vite et à moindre coût. C'est l'usage « prototype » du vibe coding créer une version d'essai pour décider.

Application : L'équipe utilise une plateforme de création par l'IA pour bâtir un prototype fonctionnel de son idée. Elle le fait évoluer par itérations, le met entre les mains de quelques utilisateurs, observe. Le prototype révèle ce qui marche et ce qui ne marche pas bien plus sûrement qu'une discussion théorique. Forte de ces enseignements, l'équipe décide en connaissance de cause : abandonner, ajuster, ou confier à de vrais développeurs une version désormais bien spécifiée. Le vibe coding lui a évité d'investir lourdement dans une idée non vérifiée.

Cas 3 : Le codeur qui accélère et délègue le répétitif

Situation : Léa développe déjà. Une part de son travail est constituée de tâches répétitives et de code sans difficulté conceptuelle, qui lui prennent du temps sans la faire progresser.

Diagnostic : Léa n'a pas besoin d'apprendre à créer elle sait. Son enjeu est la vitesse : déléguer le répétitif pour se concentrer sur la conception et les choix difficiles.

Application : Léa adopte un assistant de code intégré à son environnement. Elle lui confie le code répétitif et les premières versions, en restant celle qui conçoit, relit et décide. Elle teste systématiquement ce que l'assistant produit sa vigilance de développeuse ne baisse pas et garde la maîtrise de l'architecture d'ensemble. Léa avance nettement plus vite, et réinvestit le temps gagné dans les parties de son métier où son jugement fait la différence. Le vibe coding ne l'a pas remplacée : il l'a amplifiée.

Partie 5 : Outils, templates et checklists

Les concepts ne valent que par leur application. Cette partie regroupe les outils opérationnels que vous pouvez utiliser dès cette semaine.

5.1 Template : la carte d'un projet de vibe coding

Les éléments à poser avant de se lancer dans un projet :

1. Intention : que doit faire l'outil, exactement, et pour qui ?
2. Cas d'usage : dans quelles situations sera-t-il utilisé, y compris particulières ?
3. Enjeu : le projet est-il à faible enjeu, ou touche-t-il à des données ou systèmes sensibles ?
4. Point de départ : ai-je déjà codé, ou est-ce une première ?
5. Outil : assistant de code intégré, ou plateforme de création par l'IA ?
6. Critère de réussite : à quoi saurai-je que l'outil fonctionne et que je le maîtrise ?

5.2 Template : la grille de choix de l'outil

Pour choisir la famille d'outils selon le besoin :

Profil ou besoin	Famille d'outils indiquée
Jamais codé, idée à concrétiser	Plateforme de création par l'IA pas d'installation, application générée à partir d'une description.
Prototyper une idée vite	Plateforme de création par l'IA une version d'essai pour décider avant d'investir.
Code déjà, veut accélérer	Assistant de code intégré suggestions et génération au fil du travail.
Veut apprendre en faisant	Assistant de code intégré créer et comprendre en même temps.
Connecter l'IA à son environnement	Assistant de code + MCP du Module 25 l'IA codeur en prise avec le réel.

5.3 Checklist : avant de se lancer dans un projet

- L'idée est-elle simple, utile et à faible enjeu adaptée à mon niveau actuel ?
- Mon intention est-elle formulée clairement, complètement, sans ambiguïté ?
- L'outil choisi correspond-il à mon point de départ ?
- Le projet reste-t-il à l'écart des données sensibles et des systèmes critiques ?
- Ai-je défini un critère de réussite clair, qui inclut le test et la compréhension ?

5.4 Checklist : avant de mettre un code produit avec l'IA en usage réel

- L'outil a-t-il été testé sur des cas réels et variés, y compris des cas particuliers ?
- Ai-je compris, au moins dans ses grandes lignes, ce que le code fait ?
- Les points sensibles sécurité, données ont-ils été identifiés et vérifiés ?
- Le projet reste-t-il dans mon périmètre de maîtrise, ou faut-il l'avis d'un professionnel ?
- Saurai-je corriger ou faire évoluer cet outil le jour où ce sera nécessaire ?

5.5 Outils et ressources recommandés

- **Les assistants de code intégrés.** GitHub Copilot, Cursor, Claude Code : pour le codeur qui veut accélérer ou apprendre en faisant.
- **Les plateformes de création par l'IA.** Pour le non-codeur : créer et prototyper une application à partir d'une description, sans rien installer.
- **Le MCP du Module 25.** Pour connecter l'IA codeur à votre environnement réel fichiers, projets, outils.
- **Des bases de programmation.** Quelques notions de code, apprises ensuite, démultiplient la maîtrise de tout ce qui précède.
- **La FORMATION IA 26.** Le fil conducteur : 26 modules et 12 mois de mises à jour pour suivre un domaine qui évolue vite.

Les 7 pièges à éviter

Sept pièges classiques faussent la pratique du vibe coding. Les nommer permet de les éviter.

PIÈGE N°1 Confondre « ça marche » et « je maîtrise »

Un programme peut sembler fonctionner tout en cachant des erreurs et des failles. L'apparence de succès ne vaut pas vérification.

PIÈGE N°2 Décrire son intention de façon floue

Le vibe coding repose sur l'intention. Une consigne vague produit un résultat vague la clarté est la moitié du travail.

PIÈGE N°3 Viser le résultat parfait du premier coup

Le vibe coding est itératif. On décrit, on voit, on corrige un projet ne se juge pas sur le premier jet.

PIÈGE N°4 Sortir de son périmètre de maîtrise

Les données sensibles et les systèmes critiques relèvent de vrais professionnels. On débute sur des projets simples et à faible enjeu.

PIÈGE N°5 Accepter la boîte noire

Utiliser un code qu'on ne comprend pas du tout, c'est renoncer à pouvoir le corriger et le faire évoluer.

PIÈGE N°6 Sauter l'étape du test réel

Seul le test sur des cas variés révèle les défauts qu'un coup d'œil ne voit pas. Il n'est jamais optionnel.

PIÈGE N°7 Croire que le vibe coding remplace tout savoir

Il déplace la compétence vers l'intention et la vérification il ne supprime ni le jugement, ni l'intérêt d'apprendre.

Synthèse et plan d'action en 7 jours

Synthèse : les 10 idées-clés

1. Le vibe coding, c'est créer du code en décrivant son intention à l'IA, plutôt qu'en écrivant chaque ligne.
2. Il déplace la compétence rare : non plus la syntaxe, mais la clarté de l'intention et la rigueur de la vérification.
3. L'IA est un binôme de programmation : la qualité du résultat dépend de la qualité de l'échange.
4. Deux familles d'outils : les assistants de code intégrés, et les plateformes de création par l'IA.
5. Le vibe coding est itératif : on décrit, on voit, on corrige le dialogue converge vers le but.
6. Le code produit reste du code : il doit être testé, compris et maintenable.
7. Vibe coding et MCP se rejoignent : l'IA qui crée, et qui crée en prise avec le réel.
8. Il sert le non-codeur qui peut créer et le codeur qui peut accélérer.
9. Le piège central : confondre « cela marche » et « je maîtrise ce que j'ai construit ».
10. On reste dans son périmètre, on teste réellement, on refuse la boîte noire.

Plan d'action en 7 jours

Les 10 idées-clés ne valent que mises en mouvement. Voici un plan opérationnel sur 7 jours une heure par jour suffit.

- **Jour 1 Lundi** : repérez une idée d'outil simple, utile et à faible enjeu que vous aimeriez réaliser.
- **Jour 2 Mardi** : choisissez la famille d'outils adaptée à votre point de départ.
- **Jour 3 Mercredi** : mettez votre intention au clair, et décrivez-la précisément à l'IA.
- **Jour 4 Jeudi** : construisez une première version, puis avancez par itérations.
- **Jour 5 Vendredi** : testez votre outil sur de vrais cas, y compris des cas particuliers.
- **Jour 6 Samedi** : faites-vous expliquer ce que vous avez produit, et vérifiez que vous le comprenez.
- **Jour 7 Dimanche** : améliorez votre outil, et identifiez votre prochain projet, un cran plus ambitieux.

À RETENIR

Le Module 26 referme la formation, et il le fait sur la transformation la plus profonde : l'IA ne se contente plus d'assister votre travail, elle vous permet de créer. La dernière barrière de l'informatique « il faut savoir coder » s'abaisse. Mais le fil rouge de toute la formation tient jusqu'au bout : l'IA amplifie ce que vous êtes capable de penser, de décrire et de vérifier. Elle ne remplace pas votre jugement. Le jour où une idée à vous devient un outil concret, testé et compris, vous avez saisi l'essentiel des 26 modules.

Pour aller plus loin

Ressources commentées

- **La documentation des assistants de code.** GitHub Copilot, Cursor, Claude Code : leurs guides officiels sont la meilleure prise en main.
- **La documentation des plateformes de création par l'IA.** Pour découvrir, pas à pas, la création d'une première application sans installation.
- **Une initiation aux bases de la programmation.** Quelques notions de code, apprises après une première expérience de vibe coding, en démultiplient la maîtrise.
- **Le Module 25 Model Context Protocol.** Pour connecter l'IA codeur à votre environnement réel et la rendre bien plus utile.
- **Les Modules 2 et 3 Maîtrise des prompts.** Le vibe coding est, au fond, un art de la consigne claire : tout ce qui vaut pour le prompt vaut ici.
- **Les Modules 14 et 15 Agents IA.** Pour relier création de code et autonomie d'exécution.

Liens avec les autres modules FORMATION IA 26

Le Module 26 referme le bloc « automatisation et MCP », et la formation entière. Les modules voisins l'éclairent :

- **Module 25 Model Context Protocol** : le MCP connecte l'IA codeur à votre environnement fichiers, projets, outils.
- **Module 24 Make, Zapier, n8n** : ce que l'on crée avec l'IA peut s'intégrer à des processus automatisés.
- **Modules 2 et 3 Maîtrise des prompts** : le vibe coding est un art de l'intention claire la compétence-socle de toute la formation.
- **Modules 14 et 15 Agents IA** : créer du code et exécuter des tâches en autonomie se rejoignent.
- **Module 1 IA, LLM et ChatGPT** : la boucle est bouclée des fondations de l'IA à la création avec l'IA.

À propos de la formation IA 26

Ce guide est une introduction au Module 26 d'une formation complète de 26 modules.

FORMATION IA 26 est une formation conçue par Matthieu Jardin, fondateur d'Impact Croissance et de Marketing AI. Elle s'adresse aux dirigeants, indépendants, solopreneurs et équipes qui veulent maîtriser l'intelligence artificielle pour gagner en productivité et développer leur activité.

Format

- 26 modules vidéo, un panorama complet de l'IA générative et de ses outils.
- Formation structurée module par module, accessible à votre rythme.
- Supports pédagogiques complets et bonus exclusifs par module.
- Accompagnement sur groupe WhatsApp privé et pod LinkedIn d'entraide.
- 12 mois de mises à jour des supports offertes.

Les 26 modules en un coup d'œil

- **Modules 1 à 4** : fondations et maîtrise de ChatGPT (IA-LLM, prompts, fonctions avancées, modèles).
- **Modules 5 à 9** : le panorama des IA génératives (Gemini, Claude, Copilot, Grok, Mistral, IA émergentes).
- **Modules 10 à 13** : productivité augmentée (NotebookLM, Noota, Gamma, Napkin, recherche avancée).
- **Modules 14 à 18** : agents IA et navigateurs intelligents (Genspark, Manus, Flowith, Skywork, Perplexity).
- **Modules 19 à 21** : IA locale, privée et assistants connectés (LM Studio, Msty, Ollama, Dust, H Company).
- **Modules 22 et 23** : création visuelle avec l'IA (images et vidéos).
- **Modules 24 et 25** : automatisation (Make, Zapier, n8n) et Model Context Protocol (MCP).
- **Module 26** : IA pour codeurs (Vibe Coding).

Investissement

- **Valeur totale de la formation** : 25 475 €.
- **Votre avantage** : un seul paiement de 1 995 €.

Offre de lancement exclusive, code « IA » : -50 % (997,50 € au lieu de 1 995 €), limitée aux 10 premiers inscrits.



[Cliquez ici pour accéder à la page d'inscription](#)

Contacts et rendez-vous offert

Vous avez parcouru ce guide. Maintenant, prenons 15 minutes ensemble pour transformer cette lecture en mise en mouvement concrète.

Rendez-vous de conseil offert : 15 minutes

👉 Profitez d'un moment constructif, créatif et fructueux . Un [rendez-vous de conseil offert de 15 minutes](#), l'occasion pour étudier ensemble vos os besoins de formation en IA

✅ [Vous voulez en profiter ?](#) : 🚀 [cliquez-ici](#)

FORMATION IA 26

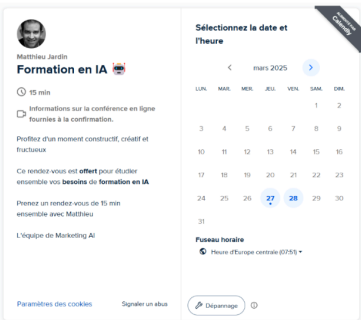
Un rendez-vous de conseil : offert

📅 🧡 Profitez d'un moment constructif, créatif et fructueux . Un [rendez-vous de conseil offert de 15 minutes](#), l'occasion pour étudier ensemble vos besoins de formation en IA

✅ [Vous voulez en profiter ?](#)

🚀 [Cliquez sur ce lien](#)





112

Matthieu Jardin



Matthieu Jardin est AI Business Developer, fondateur d'Impact Croissance et de Marketing AI. En quatre ans, il a formé plus de 1000 entrepreneurs et professionnels en IA, sensibilisé plus de 50 000 entrepreneurs, et reçu plus de 416 recommandations sur LinkedIn. Il enseigne en école supérieure et conseille les dirigeants. Sa posture est stable : transmettre d'abord, vendre ensuite ; livrer un système plutôt qu'un savoir ; rester proche du concret plutôt que du jargon.

Mes coordonnées

Email : contact@matthieujardin-formation.fr

Tel. int' : +33 6 29 19 92 87

Tel. : 06 29 19 92 87



Merci d'avoir lu ce guide.

À très vite, de l'autre côté du passage à l'action.